C++ HANDBOOK

1. INTRODUCTION TO C++

★ What is C++ ?

C++ is a high-level, general-purpose programming language. It was developed by Bjarne Stroustrup at Bell Labs in 1979 as an extension of the C language. C++ supports both procedural and object-oriented programming.

★ Why C++ ?

- Fast and efficient.
- Used in system/software development.
- Supports both high-level and low-level programming.
- Portable across platforms.
- Large standard library (STL).
- Widely used and in demand.

★ Applications of C++

- System Software (OS, Compilers)
- Game Development
- Embedded Systems
- GUI Applications
- Web Browsers
- Databases
- Finance and Trading Systems
- Scientific and Engineering Applications
- And many more...

Key Features

- ✓ High Performance
- ✓ Compiled Language
- ✓ Multi-paradigm (Procedural + OOP)
- ✓ Object-Oriented
- ✓ Platform Independent
- ✓ Rich Standard Library (STL)
- ✓ Memory Management
- ✓ Extensible and Flexible
- ✓ Widely Used

★ C++ Program Structure

```
// Preprocessor Directive
#include <iostream>

// Namespace
using namespace std;

// Main Function
int main() {
    // Code
    cout << "Hello, C++!" << endl;
    return 0;
}                    // End of main
```

★ First C++ Program

```
#include <iostream>
using namespace std;
int main() {
  cout << "Hello, C++!" << endl;
  return 0;
}
```

→ Output : Hello, C++!

★ Compilation Process

Source Code (.cpp file) → Compiler (g++) → Object Code (.o file) → Linker → Executable File (.exe / .out)

The compiler translates the source code into machine code.
The linker links all object files and produces the final executable file.

C++ HANDBOOK

2. VARIABLES AND DATA TYPES

★ What is Variable ?

A variable is a named memory location used to store data that can be changed during program execution.

★ Rules for Naming Variables

- Must start with a letter or underscore (_).
- Can contain letters, digits and underscore.
- Cannot start with a digit.
- Case sensitive (age and Age are different).
- Cannot use reserved keywords.

★ sizeof() Operator

It is used to find the size (in bytes) of a data type or variable.

Example

```
cout << sizeof(int) << endl;      // 4
cout << sizeof(float) << endl;    // 4
cout << sizeof(double) << endl;   // 8
cout << sizeof(char) << endl;     // 1
cout << sizeof(bool) << endl;     // 1
```

★ Output in C++

We use cout object to display output.

Example

```
int age = 25;
cout << "Age: " << age << endl;
cout << "Hello, C++!" << endl;
// endl moves to next line
```

★ Variable Declaration and Initialization

Examples

```
int a;          // declaration
a = 10;         // initialization
int b = 20;     // declaration + initialization
float price = 99.99f;
char grade = 'A';
bool isActive = true;
```

Always initialize variables before using them to avoid garbage values.

Primitive Data Types

Data Type	Size (Typical)	Description	Example
int	4 bytes	Integer numbers	int age = 25;
float	4 bytes	Single precision floating point	float pi = 3.14f;
double	8 bytes	Double precision floating point	double pi = 3.14159;
char	1 byte	Character	char ch = 'A';
bool	1 byte	Boolean (True / False)	bool flag = true;

★ Constants

Constants are variables whose value cannot be changed during program execution.

Example

```
const float PI = 3.14159;
const int MAX = 100;
// PI and MAX are constants
```

★ Input in C++

We use cin object to take input from the user.

Example

```
int age;
cout << "Enter your age: ";
cin >> age;
// Reads an integer from user
```

★ Escape Sequences

Sequence	Meaning	Example Output
\\n	New Line	Hello World
\t	Tab	A B
\\"	Double Quote	"C++"
\\	Backslash	C:\\Code

→ Variables are the containers and data types define the kind of data. ←

C++ HANDBOOK

3. OPERATORS AND CONTROL STATEMENTS

★ Types of Operators

Type	Description
Arithmetic	+, -, *, /, %
Relational	==, !=, >, <, >=, <=
Logical	&&, ||, !
Assignment	=, +=, -=, *=, /=, %=
Increment / Decrement	++, --
Bitwise	&, |, ^, ~, <<, >>

★ Control Statements

- Used to control the flow of execution in a program.
- Helps in decision making and looping.

★ 1. if Statement

Syntax :

```
if (condition) {
    // block of code
}
```

Example

```
int x = 10;
if (x > 5) {
    cout << "x is greater than 5";
}
```

★ 2. if-else Statement

Syntax :

```
if (condition) {
    // block of code
} else {
    // block of code
}
```

Example

```
int x = 3;
if (x % 2 == 0) {
    cout << "Even";
} else {
    cout << "Odd";
}
```

★ 3. switch Statement

Syntax :

```
switch (expression) {
case value1:
    // block of code
    break;
case value2:
    // block of code
    break;
default:
    // block of code
}
```

Example

```
int day = 2;
switch (day) {
case 1: cout << "Mon"; break;
case 2: cout << "Tue"; break;
case 3: cout << "Wed"; break;
default: cout << "Invalid";
}
```

Note :
✓ break is used to stop the execution.
✓ default is optional.

★ 1. Arithmetic Operators

Operator	Name	Example	Result
+	Addition	a + b	a plus b
-	Subtraction	a - b	a minus b
*	Multiplication	a * b	a times b
/	Division	a / b	a divided by b
%	Modulus	a % b	Remainder of a/b

★ 2. Relational Operators

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

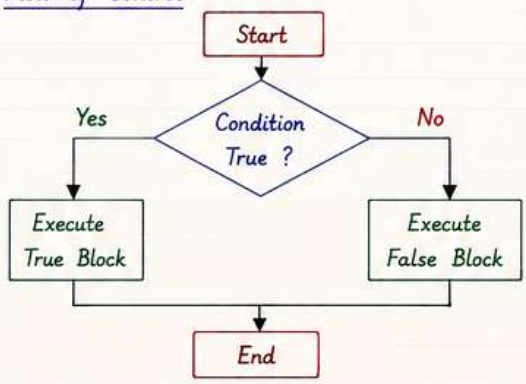
★ 3. Logical Operators

Operator	Meaning	Example
&&	Logical AND	(a > 0 && b > 0)
||	Logical OR	(a > 0 || b > 0)
!	Logical NOT	!(a > 0)

★ 4. Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

★ Flow of Control



★ Looping Statements

- for loop
- while loop
- do-while loop

C++ HANDBOOK

4. FUNCTIONS AND ARRAYS

★ What is Function ?

A function is a block of code that performs a specific task.
It helps in code reusability and makes the program modular.

★ Types of Functions

- No return value, no arguments
- No return value, with arguments
- With return value, no arguments
- With return value, with arguments

★ Function Syntax

```
return_type function_name(parameter_list) {
    // body of the function
    return value;   // optional
}
```

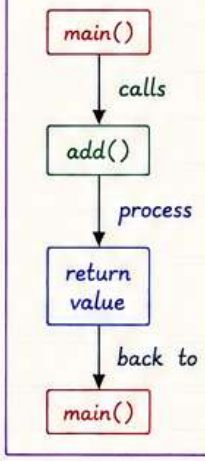
★ Function with Parameters

```
int multiply(int a, int b) {
    return a * b;
}
// function call
int product = multiply(5, 6);
cout << product;    // Output: 30
```

★ Example : Function

```
int add(int a, int b) {
  int sum = a + b;
  return value;
}
int main() {
  int x = 10, y = 20;
  int result = add(x, y); // function calll
  cout << "Sum = " << result << endl;
  return 0;
}
```

Function Flow



★ Function Overloading

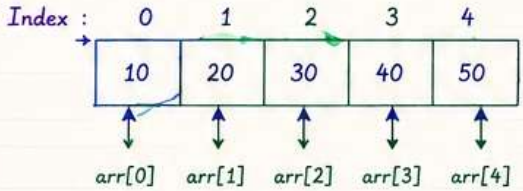
C++ allows multiple functions with same name but different parameters.

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a+b; }
int add(int a, int b, int c) { return a+b+c; }
```

★ Arrays in C++

An array is a collection of elements of the same data type stored in contiguous memory locations.

Memory Layout (1D Array)



- **1D Array**

```
int arr[5] = {10, 20, 30, 40, 50};
cout << arr[0];    // Output: 10
cout << arr[4];    // Output: 50
```

- **2D Array**

```
int mat[2][3] = { {1, 2, 3},
                  {4, 5, 6} };
cout << mat[0][1];   // Output: 2
cout << mat[1][2];   // Output: 6
```

Memory Layout (2D Array)

Row \ Column →	0	1	2
0	mat[0][0] 1	mat[0][1] 2	mat[0][2] 3
1	mat[1][0] 4	mat[1][1] 5	mat[1][2] 6

Key Points

- ✓ Functions reduce code repetition.
- ✓ Arrays store multiple values of same type.
- ✓ Array index starts from 0.
- ✓ Out of bound access may cause error.

C++ Handbook

5. Pointers and References

★ What is Pointer ?

A pointer is a variable that stores the memory address of another variable.

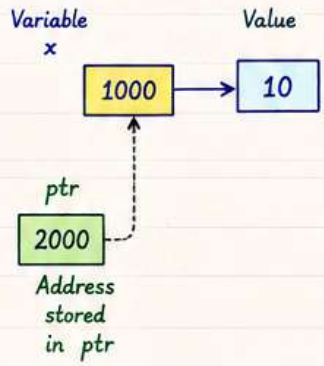
Syntax :

```
data_type* pointer_name;
// pointer declaration
```

★ Pointer Example

```
int x = 10;
int* ptr = &x;   // & is address
cout << *ptr;    // Output: 10
*ptr = 20;       // change value
cout << x;       // Output: 20
```

Memory Diagram

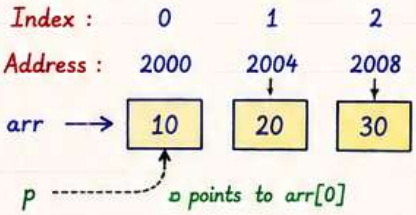


★ Pointer Operators

Operator	Name	Meaning	Example
&	Address of	Gets the address of a variable	int x; &x;
*	Dereference	Access value at an address	int* p; *p;

★ Pointer with Arrays

```
int arr[3] = {10, 20, 30};
int* p = arr;   // arr decays to pointer
cout << *(p+1); // Output: 20
```



★ References in C++

A reference is an alias (another name) for an existing variable.

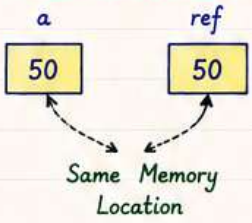
Syntax :

```
data_type& ref_name = variable_name;
```

Reference Example

```
int a = 10;
int& ref = a;  // ref is reference of a
ref = 50;      // changes a
cout << a;     // Output: 50
```

Reference Diagram



★ Pointer vs Reference

Feature	Pointer	Reference
Memory	Stores memory address	Alias of existing variable
Can be null ?	Yes	No
Can be changed ?	Yes, can point to different variables	No, cannot be changed once initialized
Syntax	int* p;	int& r = x;
Size	Depends on system (4 or 8 bytes)	Same as variable
Use	Dynamic memory, arrays, data structures	Function parameters, safer access

★ Dynamic Memory Allocation (New and Delete)

Using new

```
int* p = new int;   // allocate memory
*p = 100;           // store value
cout << *p;         // Output: 100
```

new allocates memory from heap at runtime.

Using delete

```
delete p;       // free memory
p = nullptr;    // good practice
```

Important

- ✓ Always delete what you allocate.
- ✓ Avoid memory leaks.
- ✓ Use nullptr instead of NULL.

C++ HANDBOOK

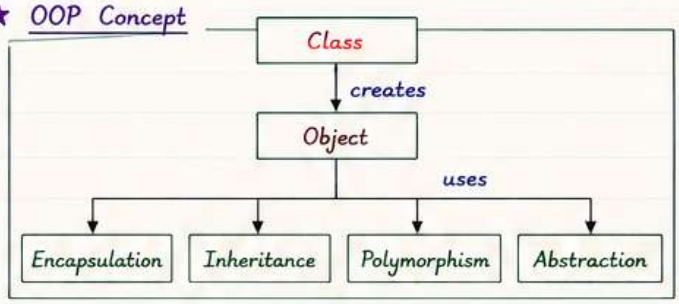
6. OOP BASICS

★ What is OOP ?

OOP (Object Oriented Programming) is a programming paradigm that uses "objects" and "classes" to design programs.

It makes code modular, reusable and easy to maintain.

★ OOP Concept



★ 1. Class and Object

Class : A bleuprint or template that defines properties and behaviors.

Object : An instance of a class.

★ 2. Constructor and Destructor

Constructor

- Special member function.
- Has same name as class.
- No return type.
- Called automatically when object is created.

Destructor

- Special member function.
- Has ~ before class name.
- No return type.
- Called automatically when object is destroyed.

★ Example : Class and Object

```
class Student {
 public:
    string name;
    int age;

    void display() {
      cout << name << " is " << age << " years old.";
    }
};

int main() {
   Student s1;          // object of class Student
   s1.name = "Aman";
   s1.age  = 25;
   s1.display();
   return 0;
}
// Output : Aman is 25 years old.
```

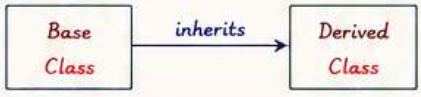
★ 3. Access Specifiers

Specifier	Access Within Class	Access Outside Class	Inherited
public	Yes	Yes	Yes
private	Yes	No	No
protected	Yes	No	Yes

By default, members of a class are private.

★ 4. Inheritance

Inheritance allows a class to acquire properties and behaviors of another class.



Types of Inheritance

- Single
- Multilevel
- Multiple
- Hierarchical
- Hybrid

★ 5. Polymorphism (Many Forms)

Polymorphism allows objects to take many forms.

It is achieved by :

- Function Overloading (compile-time)
- Function Overriding (run-time using virtual)

★ 6. Abstraction

Abstraction means showing only essential features and hiding implementation details.

It is achieved using :

- Abstract Class
- Pure Virtual Function

Key Point :
OOP helps in writing better, scalable and easy to maintain programs.

★ Example : Simple BankAccount Class

```
class BankAccount {
 private:
    double balance;
 public:
    BankAccount(double b = 0) { balance = b; }   // Constructor
    void deposit(double amount) { balance += amount; }
    void withdraw(double amount) { balance -= amount; }
    double getBalance() { return balance; }
    ~BankAccount() { cout << "Account destroyed." << endl; } // Destructor
};
```

```
int main() {
    BankAccount acc(1000.0);
    acc.deposit(500.0);
    acc.withdraw(200.0);
    cout << "Balance: " << acc.getBalance();
    return 0;
}

// Output : Balance: 1300
```

Think → Design → Code → Test → Improve

COMMENT C++

TO GET

COMPLETE PDF

SAVE | SHARE | FOLLOW

aman_views